Autonomous Data-Analysis Agent — System Architecture

1. Overview

The Autonomous Data-Analysis Agent takes one or more raw datasets and a plain-English question, in any supported language, and returns a verified answer: the reasoning, code, and corrections that produced it, the specific rows and columns behind it, an honest statement of how far the answer can be trusted given the data's own quality, and an explicit note whenever the question asked for something the dataset alone can't actually support.

It is a genuine agent, not a single-shot code generator. It runs a real loop — plan, act, observe, retry — and goes further than that basic loop in five specific ways: it re-plans when execution reveals that its own assumptions were wrong, it remembers across a session so a follow-up question builds on prior work instead of starting cold, it checks its own cache before doing new work so repeated questions are nearly free, it checks its own answers against each other — within a session always, and across sessions analyzing the same dataset if the user opts in — for quiet contradictions, and it stays available even when its primary model provider isn't, falling back to local inference rather than simply failing.

Two model tiers sit over a local orchestration and governance layer:

- A **Router** classifies each question as trivial or not, sending trivial ones straight to execution and saving a full planning call.
- A **Planner** turns a non-trivial question into a structured, typed plan — or, when the question is genuinely ambiguous, into a small, budgeted set of competing plans representing the plausible readings.
- An **Executor** turns each plan step into code, checks a cache before generating anything, statically checks the code it does generate, runs it inside a policy-governed sandbox against pandas or DuckDB, reads the result, retries on error, and flags the orchestrator when a step succeeds but reveals something the plan didn't anticipate.
- A **local orchestration and governance layer** — a state-machine orchestrator, an embedded vector store, a two-tier cache, sandboxed dual-backend execution, and a web UI — coordinates all of this, enforces cost budgets at both the question and session level, checks answers for internal consistency, and runs on an elastic pool of isolated, individually sandboxed workers behind per-user isolation and rate limiting.
- An **evaluation harness**, run offline, exercises the exact same pipeline against a fixed, coverage-tracked benchmark set, and turns negative user feedback into an automatically drafted candidate fix rather than an open-ended investigation.

2. Design principles

Eight principles govern every decision in this architecture.

Show the work. Every code attempt, every error, every retry is visible, not hidden behind a spinner. The value of the agent isn't just the final number — it's that the process is auditable.

Verify before presenting. "The code ran without error" and "this answers the question" are different claims, and the system treats the second as the actual bar, with an explicit verification step rather than conflating the two.

Ground in the real schema, never let a model guess it. Column names, dtypes, value ranges, and the relationships between tables all come from actually examining the data — profiled, sample-tested, and validated — never assumed by a language model.

Bound every loop. Retries, re-plans, and hypothesis forking each have a hard cap and a defined escalation path. What a single generated attempt is allowed to do is bounded by three independent layers, not one.

Nothing leaves the machine unmasked. The only network egress in the whole system is the model API calls, and personally identifiable information is masked before it ever reaches them, by default, without the user having to think about it.

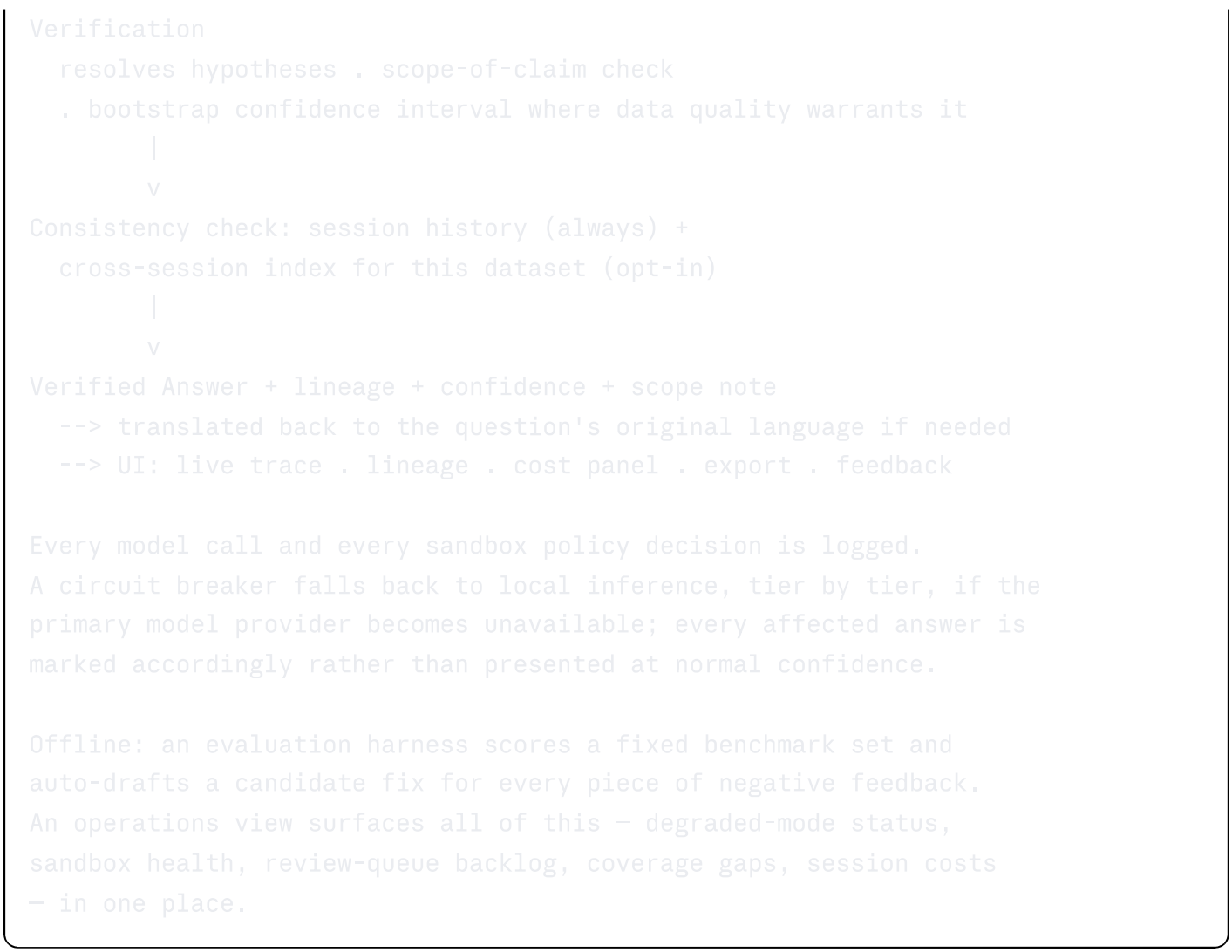
Measure the bet, don't just assert it. Every model call is timed and token-counted. A fixed, coverage-tracked benchmark set reports real accuracy and retry numbers. Nothing about how well this system performs is left as an impression.

Don't re-derive what's already verified. A step already computed for this exact dataset is reused, not redone. An answer that should logically agree with an earlier one in the same session — or, with consent, in another session against the same data — is checked against it.

Every flag needs a resolution path, not just a warning. A system that notices something is wrong and only tells the user about it has done half the job. A cache match that might be subtly wrong is always re-verified by fresh execution before being trusted. A contradiction between two answers is checked, not silently ignored. Negative feedback gets an automatically drafted fix attached, not just a queue entry. A limitation of what the system can know is stated explicitly at the moment it's relevant, not left as an assumption the user has to already know to account for.

3. High-level architecture





4. Components

4.1 Ingestion & governance

Runs entirely locally; nothing here involves a model call. This is the stage that gates everything downstream, and it does five things on every upload and every question.

Modality gate. Before anything else, the upload is checked against a small set of supported tabular formats — CSV, Excel, Parquet, JSON. Anything that doesn't parse as one of these (a scanned document, an image, free-form text) is rejected immediately with a specific, named reason, rather than being forced through the rest of the pipeline and failing unpredictably somewhere deeper in.

Profiling. For each table, each column is profiled: name, inferred dtype, null percentage, cardinality, numeric min/max where relevant, and a small sample of values. This profile — never the raw data itself — is what the rest of the system reasons about.

PII detection. Every column is scanned for common PII patterns — emails, phone numbers, government-ID-shaped numbers, personal names. Flagged columns have their sample values

masked by default in anything that leaves this stage; the underlying data used for actual computation is untouched, only what's shown to the model tier is affected.

Data-quality scan. Beyond nulls, the scan flags outliers, type mismatches, inconsistent formats, and duplicate rows, producing a human-readable summary shown to the user immediately and passed to the Planner so a plan can account for known issues from the start.

Multi-table join inference. When more than one table is loaded, candidate join keys are identified by name similarity, value overlap, and cardinality signals, and medium-confidence candidates are then sample-validated — an actual trial join run against a sample of real data, checked for sane match rate and row-count behavior — before being trusted automatically. Only genuinely ambiguous or too-small-to-sample candidates are surfaced to the user, and once answered, that decision is persisted so the same dataset never asks about the same join twice.

Language handling. The question's language is detected before it reaches the Router. Supported non-English questions are translated to English for the entire pipeline, and the final answer — along with the assumptions and scope note attached to it — is translated back into the question's original language before being shown. A language outside what's supported gets the same clear, immediate rejection treatment as an unsupported file format, rather than being silently forced through.

Every dataset also gets a content fingerprint computed once at this stage — a hash of its actual content — used throughout the system to scope the cache, the session, and the cross-session consistency index, so a re-uploaded but changed file never collides with anything stale.

4.2 Vector store — schema grounding, session memory, caching & consistency

An embedded vector store running locally, holding four related but distinct collections.

Schema retrieval. For datasets too wide to comfortably fit their full profile in a prompt, each column is embedded as its own document (name, dtype, quality flags, masked samples), and a question retrieves only the top few relevant ones rather than the Planner seeing every column in the dataset.

Session memory. Prior question/plan/answer triples within a session are stored so a follow-up question — "now break that down by quarter" — can build on an earlier result instead of restarting from the raw dataset.

Semantic cache index. Every cached step's description is embedded here, keyed to its dataset fingerprint, so a step that's semantically similar to (but not textually identical to) one already computed can be found even when the exact-match cache misses.

Cross-session consistency index. Opt-in, off by default, and deliberately minimal: it stores only a dataset's content fingerprint, a normalized signature of which columns and aggregation

a verified answer's lineage touched, the resulting value, and a timestamp — never the question text, never a user or session identifier. This is enough to detect that two separate analyses of the same dataset should logically agree, without either party learning anything about what the other asked.

4.3 Router

One cheap, fast model call per question, classifying it as `direct` — answerable in a single step, no real planning needed — or `planned`, which proceeds to the full Planner. A wrong classification is inexpensive to recover from: a `direct` route that turns out to need more steps simply surfaces as a plan-level issue and escalates through the same re-planning path any other one takes. The Router is also the first tier to fall back to local inference if the primary model provider is unavailable, since a routing misclassification is the cheapest kind of mistake anywhere in the system to absorb.

4.4 Planner

Receives the question and the relevant schema context and returns a structured, validated plan via function calling — never free text the Executor would have to parse. Each plan carries a confidence score reflecting how directly the question maps to a single interpretation: high confidence returns one plan, medium confidence returns two competing ones, and genuinely low confidence returns up to three, subject to a token budget that can drop the weakest candidates before execution starts, or stop executing the others early if one shows a clearly dominant signal partway through.

Every plan step also carries a wave number, computed from its dependencies at planning time, so the orchestrator knows exactly which steps can run concurrently without having to infer it later — and a target backend, pandas or DuckDB, based on data size and whether the step needs a cross-table join.

When execution reveals a plan-level problem — a step that succeeds technically but exposes something the plan didn't anticipate, as opposed to a step that simply errors — the Planner is called again with the completed work and a description of what was found, and revises only the remaining steps rather than starting over.

Every model call in the system, including the Planner's, goes through a shared provider-resilience layer: primary calls go to the hosted API; a circuit breaker trips to local inference after sustained failures. The Planner is the last tier to fall back, since its reasoning quality matters the most — and when it does, every affected plan and the answer that comes from it is explicitly marked as produced under degraded conditions, shown as such in the interface rather than presented at ordinary confidence.

4.5 Executor

The fast, repeated inner loop, and the component that runs by far the most often. For each plan step:

1. **Cache check.** An exact-match lookup first (identical dataset fingerprint and step description — an instant hit reuses the whole verified result), then a semantic lookup (a similar-enough step found by embedding similarity) — a semantic hit reuses only the *code*, never the stored result; that code is always executed and verified fresh, exactly as if it had just been generated, so a fuzzy match can only ever save a generation call, never produce an unchecked answer.
2. **Generate**, on a cache miss — write code targeting the step's chosen backend, given the running transcript of everything tried so far for this step.
3. **Statically check** the generated code against a fixed rule set before it goes anywhere near real execution, rejecting disallowed constructs outright.
4. **Execute** inside the sandbox (4.6).
5. **Read** stdout, any traceback, and the resulting value.
6. **Decide**: success advances to the next step and caches the result; an error retries with the failure appended to context, up to a fixed cap; a success that reveals a plan-level problem routes to re-planning instead of either.

Lineage — rows in and out, and which columns were touched — is captured on every attempt, which is what lets the final answer be traced back to specific rows and columns rather than just the code that produced it.

4.6 Sandboxed code execution

Code never runs in the application's own process. Execution goes through a persistent kernel — the same dataframe and intermediate variables need to survive across steps and across retries within a step, which is why each Executor call is effectively one cell execution against a kernel that's been running since the session started, rather than a fresh interpreter spun up per call. That kernel runs against either pandas or DuckDB depending on the step's chosen backend.

The kernel itself runs inside a policy-governed sandbox: deny-by-default network access (generated analysis code has no legitimate reason to reach the network at all), filesystem writes permitted only to a scratch directory used for saving charts, and hard limits on memory and wall-clock time per execution, enforced at the operating-system level rather than hoped for by the sandboxing code. Every allow and deny decision the policy makes is logged. A startup health check confirms this sandbox layer is actually available and correctly configured before a session relies on it; if it isn't, execution falls back automatically to an alternative,

container-based isolation implementation with an equivalent — if less refined — set of restrictions, logged and shown quietly in the interface rather than failing the session outright.

What a single generated attempt is allowed to do is therefore checked three separate times, by three independent mechanisms that don't assume the others are sufficient on their own: the static check before execution even begins, the sandbox's runtime policy while it runs, and the resource limits bounding how long or how much it can consume regardless of what it's doing.

4.7 Caching layer

The exact-match tier is a local key-value store, keyed on a hash of the dataset's content fingerprint, the step's description, and the target backend, storing the full successful result. The semantic tier lives in the vector store (4.2) and is checked second, since a hit there still costs a fresh sandbox execution rather than being an instant return. A dataset that's re-uploaded but has actually changed produces a new fingerprint automatically, so nothing stale is ever silently reused.

4.8 Orchestrator

The state machine coordinating everything above, built explicitly as a graph rather than a hand-rolled loop, specifically because the retry and re-planning cycles this system depends on are naturally cycles in a graph, not linear control flow.

Its nodes carry a question from routing through planning, wave-by-wave execution, retry, re-planning, verification, and consistency checking. Steps within a wave dispatch concurrently rather than one at a time, so a plan with real width — several independent aggregations feeding into a later combining step, for instance — benefits from parallelism rather than only the narrow case of one free-standing early step. Where a question forks into multiple hypotheses, each executes with its own step index and its own retry budget, tracked independently.

Two budgets are enforced here, at two different scopes: a per-question token budget bounding how much a single ambiguous question's hypothesis fan-out can cost, and a separate session-level budget tracking cumulative spend across every question asked so far in the session, with the interface warning as it's approached and requiring explicit confirmation to continue past it.

After every verified answer, the orchestrator checks it against this session's own prior answers with overlapping lineage — always, with no opt-in needed, since this check never leaves the session — and, if the user has opted in for this dataset, against the anonymized cross-session index as well. A genuine mismatch either way is surfaced to the user as a flag, not silently resolved in either direction, since a real difference in filters or assumptions between two questions can legitimately explain it.

4.9 User interface

A web interface built around one core interaction — upload, ask, watch the plan and each step's code, output, and retries stream in live as they happen, then see the verified answer — surrounded by the supporting views the rest of this architecture produces: a pre-flight panel showing the data-quality and PII summary immediately after upload, before any question is asked; a lineage view attached to every answer, in plain language; a computed confidence range shown alongside the answer whenever the data quality behind it actually warrants one; a scope note when a question asked for something beyond what the dataset can support; a cost and latency summary for the run just completed, alongside what an equivalent run using only the stronger model tier would have cost, for comparison; side-by-side panels when a question was ambiguous enough to fork into multiple hypotheses, each showing its own provisional result as soon as it's ready rather than the whole interface waiting on the slower of the two; a quiet, non-alarming notice whenever a run used a fallback model or a fallback sandbox rather than the primary path; an export action producing a re-runnable notebook of the whole session; thumbs-up and thumbs-down feedback on any answer; a one-time opt-in prompt for cross-session consistency checking, per dataset; and a follow-up input that stays active after an answer, so a session continues rather than ending after one question.

4.10 Multi-user support & deployment

A lightweight auth layer assigns each user their own isolated session — their own slice of the vector store, their own cache scope, their own running cost total — so nothing about one user's data, questions, or spend is visible to another. Requests are rate-limited per user, a distinct control from the token-cost budget: the cost governor bounds how much a session is allowed to spend, while rate limiting bounds how *often* a user can issue requests at all, protecting the system from simple abuse or runaway automation independent of cost.

Sandbox workers are drawn from an elastic pool of individually isolated instances rather than one shared sandboxing layer trying to serve everyone at once: a baseline is held pre-warmed and ready, on-demand instances are created above that baseline up to a hard ceiling reflecting actual host capacity, idle on-demand instances are torn down after a short idle period, and a request arriving when the pool is at its ceiling waits in a queue with a timeout rather than failing outright. The whole application is containerized, with the pool's baseline and ceiling configurable per deployment.

5. Observability, evaluation & operations

5.1 Cost & latency instrumentation

Every Router, Planner, Executor, and verification call is timed and token-counted into a per-run log, which feeds both the per-run cost summary shown to the user and the running session-level total the cost governor checks against. The sandbox's policy audit log — every allow and deny decision made during a run — is folded into the same store as a distinct event

type, useful both as a security record and as a signal for whether the Executor's prompting needs adjustment if a particular kind of denial keeps recurring.

5.2 Evaluation harness

A fixed, developer-curated set of dataset/question/expected-answer triples, each tagged with a category — aggregation, filter, join, time-series, ambiguous/multi-hypothesis, large-data, PII-masked — run in batch through the exact same pipeline the live system uses, not a separate reimplementation. Results report overall pass rate, per-category coverage with thin categories flagged explicitly as gaps, and retry/cost statistics. Runs produced under a degraded-mode fallback are excluded from this regular scoring, since they aren't representative of the system operating normally.

The set grows from two directions: a developer curating cases deliberately to fill coverage gaps, and real usage — when a user marks a verified answer as correct, that triple becomes a candidate case for a developer to review and promote. Negative feedback takes a different path: every thumbs-down is logged with its full transcript, and the harness automatically follows up with an independent re-derivation — a fresh planning-and-execution run with no access to the original flagged transcript. If the independent answer agrees with the one flagged, the entry is marked for the developer to investigate why it was flagged rather than assumed wrong; if it disagrees, the fresh answer becomes an auto-drafted candidate correct answer attached to the entry, reducing the developer's actual task to a single confirm-or-reject decision rather than an open-ended investigation.

5.3 Operations view

A single consolidated surface — separate from the end-user interface — pulling together everything an operator needs to know that no single user-facing panel shows on its own: whether the system is currently running on its primary model provider or a fallback, and for how long; the sandbox runtime's current health and which implementation is active; the size of the pending review queue and how many entries have an auto-drafted fix awaiting confirmation; which benchmark categories currently show a coverage gap; and aggregate cost and rate-limit statistics across all active sessions. This exists specifically because degraded-mode status, sandbox health, review-queue backlog, and coverage gaps are each individually handled elsewhere in the system, but nowhere else are they visible together — and an operator deciding whether the system is healthy needs exactly that combined view.

6. End-to-end data flow

1. A user authenticates into an isolated, rate-limited session and uploads one or more datasets. Ingestion checks the format, profiles the data, scans for PII and quality issues, and infers and sample-validates joins across tables.

2. The quality and PII summary is shown immediately, and the user is asked whether to opt in to anonymized cross-session consistency checking for this dataset.
3. The user asks a question, in any supported language. It's translated to English if needed, and relevant schema context — and any relevant prior session context — is retrieved.
4. The Router classifies the question as direct or planned.
5. On the planned route, the Planner returns one plan, or a confidence-scaled, budgeted set of competing plans.
6. The orchestrator walks the plan wave by wave. For each step, the Executor checks the cache, generates code on a miss, statically checks it, and executes it inside the sandbox against the chosen backend.
7. Errors retry up to the cap; a plan-level issue routes back to the Planner for a partial re-plan; a clean success records lineage and writes to the cache.
8. Once a hypothesis's steps are all complete, verification checks the synthesized answer against the question, resolves multiple hypotheses if there were any, checks whether the question asked for something beyond what the dataset supports, and runs a local sensitivity pass for a confidence interval if the answer's lineage touched flagged data.
9. The answer is checked against this session's history, and, if opted in, against the anonymized cross-session index; a genuine mismatch is flagged either way.
10. The answer, translated back to the question's original language if needed, along with its lineage, confidence interval, scope note, any charts, the full trace, and the cost summary, renders in the interface, with export and feedback available, and the session stays open for a follow-up.
11. Throughout, every model call and every sandbox policy decision is logged; if the primary model provider becomes unavailable, a circuit breaker shifts the affected tiers to local inference, and every answer produced that way is marked accordingly.
12. Offline and continuously: the evaluation harness scores the benchmark set, thumbs-down feedback triggers an automatic independent re-derivation, and the operations view keeps all of this visible in one place.

7. Data contracts

```python
```

```python
class ColumnProfile(BaseModel):
    name: str
    dtype: str
    null_pct: float
    cardinality: int
    sample_values: list[str]
    pii_detected: bool
    quality_flags: list[str]

class JoinCandidate(BaseModel):
    left_table: str
    left_column: str
    right_table: str
    right_column: str
    confidence: float
    sample_validated: bool
    auto_confirmed: bool
    user_confirmed: bool | None

class DatasetProfile(BaseModel):
    tables: dict[str, list[ColumnProfile]]
    row_counts: dict[str, int]
    join_candidates: list[JoinCandidate]
    quality_summary: str
    content_fingerprint: str

class PlanStep(BaseModel):
    id: int
    description: str
    depends_on: list[int]
    wave: int
    backend: Literal["pandas", "duckdb"]

class AnalysisPlan(BaseModel):
    hypothesis_id: int
    steps: list[PlanStep]
    assumptions: list[str]
    confidence: float

class StepAttempt(BaseModel):
    step_id: int
```

```
    hypothesis_id: int
    attempt_number: int
    code: str
    stdout: str
    error: str | None
    result_repr: str | None
    chart_path: str | None
    rows_in: int
    rows_out: int
    columns_touched: list[str]
    plan_issue_detected: str | None
    static_check_rejected: bool
    policy_denied: bool
    policy_denial_reason: str | None
    cache_match_type: Literal["exact", "semantic", "none"]
    degraded_mode: bool
    sandbox_runtime: Literal["primary", "fallback"]

class CallMetrics(BaseModel):
    tier: Literal["router", "planner", "executor", "verifier"]
    tokens_in: int
    tokens_out: int
    latency_ms: int
    degraded_mode: bool

class PolicyEvent(BaseModel):
    step_id: int
    decision: Literal["allow", "deny"]
    resource: str
    reason: str

class ConsistencyFlag(BaseModel):
    scope: Literal["session", "cross_session"]
    overlap_columns: list[str]
    description: str

class ConfidenceInterval(BaseModel):
    point_estimate: float
    low: float
    high: float
    basis: str
```

```python
class ReviewQueueEntry(BaseModel):
    answer_id: str
    original_transcript: list[StepAttempt]
    independent_answer: str
    agrees_with_original: bool
    candidate_correct_answer: str | None
    status: Literal["pending_confirmation", "confirmed_bug", "confirmed_correct"]

class Feedback(BaseModel):
    answer_id: str
    rating: Literal["up", "down"]
    promoted_to_benchmark: bool
    added_to_review_queue: bool

class AgentState(TypedDict):
    session_id: str
    user_id: str
    dataset_profile: DatasetProfile
    question: str
    question_language: str
    route: Literal["direct", "planned"]
    plans: list[AnalysisPlan]
    current_step_index: dict[int, int]
    transcript: list[StepAttempt]
    retry_count: dict[int, int]
    replan_count: int
    hypothesis_budget_tokens: int
    active_hypotheses: list[int]
    session_cost_budget_tokens: int
    session_cost_spent_tokens: int
    cross_session_consistency_opt_in: bool
    call_log: list[CallMetrics]
    policy_log: list[PolicyEvent]
    final_answer: str | None
    lineage: str | None
    confidence_interval: ConfidenceInterval | None
    consistency_flags: list[ConsistencyFlag]
    scope_note: str | None
    degraded_mode: bool
    verified: bool
```

8. Error handling & retry policy

Situation	Handling
Code raises a runtime exception	Traceback appended to context, Executor retries up to the cap
Code runs but returns an empty or wrong-shape result	Soft failure, retried with a hint
Execution exceeds the timeout	Sandbox terminates the process, retried with a note that the prior approach was too slow
Static check rejects generated code	Fed back as a constraint violation, counts against the retry cap
Sandbox policy denies an attempted action	Fed back identically to a static-check rejection; logged as a policy event regardless
Semantic cache hit produces a wrong result on fresh execution	Handled as an ordinary failure — Executor regenerates from scratch
Retry cap exhausted on a step	Step marked failed; failure surfaced with attempted code and errors shown
Plan-level issue detected	Routed to the Planner to revise remaining steps, up to a fixed re-plan cap
PII detected on upload	Masked by default in anything sent to the model tier
Join candidate fails sample validation, or tables too small to sample	Surfaced once; the decision is persisted
Dataset exceeds the in-memory threshold	Sandbox selects DuckDB automatically
Two hypotheses disagree after verification	Both shown, with the interpretive difference stated
Hypothesis execution would exceed the per-question budget	Lower-confidence hypotheses dropped before execution
Session cost budget approached or exceeded	Interface warns near the limit; confirmation required past it

Situation	Handling
Per-user rate limit exceeded	New requests held or rejected with a clear wait time, independent of cost budget
Answer inconsistent with a prior one this session	Flagged to the user
Answer inconsistent with the opt-in cross-session index	Flagged, fully anonymized
Sandbox pool at its hard ceiling	Request queued with a timeout; interface shows a busy state
Answer's lineage touches quality-flagged columns	A local bootstrap sensitivity pass computes an actual confidence range
Question asks for something beyond what the dataset can support	Verification attaches an explicit scope note
Upload isn't tabular, or question language isn't supported	Rejected immediately with a specific, named reason
Primary model provider fails or times out repeatedly	Circuit breaker shifts to local-inference fallback, Router first, Executor next, Planner last; every affected attempt marked accordingly
Primary sandbox runtime fails its health check	Execution falls back to the secondary isolation implementation automatically
User marks an answer thumbs-down	Logged to the review queue; independent re-derivation runs automatically
Plan fails schema validation	One re-prompt with the validation error attached
Benchmark run regresses pass rate or opens a coverage gap	Flagged in harness output

9. What "verified" actually means

A verified answer has passed three distinct checks, not one. First, a dedicated call confirms the synthesized answer actually addresses the question that was asked — "the code ran without error" is never treated as sufficient on its own. Second, where a question was ambiguous

enough to produce more than one plan, the same pass compares the resulting answers and either identifies one as the clearly better-supported reading or presents both with the specific ambiguity stated in plain terms. Third, a scope-of-claim check looks at whether the question asked for something the dataset alone can't actually establish — a comparison to an external figure not present in the data, a causal claim, a prediction beyond the observed range — and attaches an explicit note when it did, so "verified" is never mistaken for "independently fact-checked against the world."

Where an answer's lineage touches columns carrying data-quality flags from ingestion, a fourth step runs: a local statistical sensitivity pass that resamples the affected rows under a few reasonable treatments of the flagged values and recomputes the answer each time, reporting the resulting spread as an actual computed range rather than a vague caveat. An answer whose lineage touches no flagged columns skips this entirely and reports no range — the computation only happens, and only costs anything, when there's something real to calibrate against.

10. Tech stack

Layer	Choice	Role
Fast model tier	A fast, cost-efficient hosted model	Router classification; the Executor's generate-execute-retry loop
Strong model tier	A stronger hosted reasoning model	Planning, re-planning, multi-hypothesis output, independent re-derivation
Provider resilience	A shared model-client abstraction with a circuit breaker	Falls back to local inference, tier by tier, on sustained provider failure
Local-inference fallback	A self-hosted model runtime with GPU passthrough	Keeps the system available in degraded mode during a provider outage
Orchestration	A graph-based state-machine framework	Session- and user-aware coordination of routing, planning, execution, retries, and re-planning
Vector store	An embedded vector database	Schema retrieval, session memory, semantic caching, opt-in cross-session consistency
Exact-match caching	A local key-value store	Direct reuse of verified step results
Code execution protocol	A persistent kernel process	Stateful execution against pandas or DuckDB across steps and retries
Sandbox runtime	A policy-governed sandbox, with a container-based fallback	OS-level, deny-by-default isolation with a full audit log, and a tested fallback if unavailable
Large/multi-table execution	An embedded SQL engine	Handles larger-than-memory data and cross-table joins directly
Static code analysis	AST-based rule checking	Rejects disallowed constructs before execution is even attempted
PII detection	A dedicated PII-scanning library	Flags and masks PII before it reaches the model tier
Data handling	A dataframe library and its numeric foundation	Loading, profiling, computation, statistical resampling

Layer	Choice	Role
Translation	A language-detection and translation layer	Non-English questions in and answers back out
Charting	A plotting library	Generated visualizations
Report export	A notebook-format writer	Downloadable, re-runnable session output
Interface	A Python-native web app framework	Upload, question, live trace, lineage, confidence, cost, export, feedback
Auth & rate limiting	Token-based session assignment with per-user limits	Isolation and abuse protection under multiple users
Deployment	Containerization, an elastic sandbox pool	Bounded, scalable, policy-isolated resource use under concurrent sessions
Data validation	A schema-validation library	Every contract in Section 7
Observability	Lightweight local logging	Per-call metrics, cost governance, sandbox audit events
Evaluation	An internal batch-scoring script	Coverage-tracked benchmarking, auto-drafted review queue
Operations	A consolidated internal view	Fallback status, sandbox health, review backlog, coverage gaps, cost, in one place

11. Running it locally and deploying

Single-user, local: provision credentials for the primary model provider, install and configure the sandbox runtime with a policy defining its network, filesystem, and resource restrictions, install the Python dependencies for orchestration, the vector store, both execution backends, the web interface, and PII detection, then launch the application. A first run should include uploading a small real dataset and asking a genuinely simple question, to confirm the full loop — plan, execute, verify — works end to end before relying on anything more advanced.

Multi-user deployment: containerize the application, configure the sandbox pool's baseline size and hard ceiling against actual available host capacity, set up authentication and per-user rate limits, confirm session isolation holds across the vector store, cache, and cost governor, and set a default per-session cost budget. Run the evaluation harness as a matter of routine

after any change to the Planner's or Executor's prompts or to the retry and re-planning logic, and check the operations view regularly rather than only when something visibly breaks.

12. Scope and assumptions

What remains here is a genuine boundary of what this system is built to do, not an oversight.

It analyzes exactly the data it's given, and only that data — "verified" means internally consistent and well-supported by the dataset provided, with an explicit note whenever a question reaches beyond that, not an independent check against outside facts, because no system built to analyze exactly the data it's handed can honestly claim to do more than that.

It supports a defined, explicit set of tabular formats and a defined, explicit set of languages, and treats anything outside those as an immediately communicated boundary rather than an attempt at partial, unreliable handling — a clear rejection is more honest, and more useful, than a best-effort guess at something the system wasn't built for.

It stays available during a primary-provider outage by falling back to a materially weaker local model, and it says so plainly whenever that's happening rather than presenting a degraded answer with the same confidence as a normal one — availability under real conditions was judged more valuable than silence about the trade-off that comes with it.

And it is, deliberately, a system that shows its reasoning rather than one that asks to be trusted blindly — every principle in Section 2 exists in service of that, and every component in Section 4 is a specific, concrete answer to how.